

# Introducción y Aplicación del Machine Learning (ML) en el Ámbito de la Ciberseguridad

Bienvenidos a esta sesión, donde abordaremos los fundamentos del **Machine Learning (ML)** y exploraremos cómo esta rama de la Inteligencia Artificial se ha convertido en una herramienta indispensable para fortalecer las defensas contra las amenazas cibernéticas.

---

## 1. Fundamentos del Machine Learning (ML)

El **Machine Learning** o **Aprendizaje Automático** es una rama de la inteligencia artificial que permite a las máquinas **aprender de los datos** y mejorar sus predicciones o decisiones sin ser explícitamente programadas para cada tarea específica.

A diferencia de la programación tradicional, donde los desarrolladores escriben reglas y lógicas específicas, el ML utiliza algoritmos que aprenden y se adaptan a partir de los patrones que encuentran en grandes volúmenes de datos.

### 1.1. Tipos de Aprendizaje Centrales

Existen, básicamente, dos enfoques principales en el Machine Learning:

- **Aprendizaje Supervisado:**
  - **Definición:** El algoritmo se entrena utilizando un **conjunto de datos etiquetados**. Esto significa que cada ejemplo en el conjunto de datos está emparejado con la **respuesta correcta** (la "etiqueta").
  - **Objetivo:** Aprender un modelo que pueda hacer **predicciones precisas** para datos nuevos que nunca ha visto.
- **Aprendizaje No Supervisado:**
  - **Definición:** El algoritmo trabaja con **datos no etiquetados**.

- **Objetivo:** Descubrir **patrones ocultos** o **agrupaciones (*clustering*)** en los datos sin tener respuestas predefinidas.

## 1.2. Conceptos Metodológicos Clave

Para que un modelo de Machine Learning funcione de manera efectiva, se deben dominar dos procesos esenciales:

- **Extracción de Características (*Feature Extraction*):** Este es el proceso de identificar las **variables más relevantes** o las características de los datos. La calidad y pertinencia de estas características son cruciales, ya que ayudan a **mejorar el rendimiento** y la precisión del modelo.
  - **Entrenamiento del Modelo (*Model Training*):** Es el proceso de **ajustar los parámetros de un algoritmo** de Machine Learning de forma que el modelo aprenda de los datos. Esto implica alimentar al modelo con datos de entrenamiento, permitiéndole aprender y mejorar su precisión de manera iterativa.
- 

## 2. Aplicaciones Estratégicas del Machine Learning en Ciberseguridad

El Machine Learning se ha convertido en una herramienta indispensable en el arsenal contra las amenazas cibernéticas, gracias a su capacidad para procesar y aprender de grandes conjuntos de datos a una velocidad y precisión que superan ampliamente las capacidades humanas.

Su aplicación en ciberseguridad se centra en diversos frentes:

### 2.1. Detección de Anomalías y Comportamientos Sospechosos

Este es el enfoque más clásico y fundamental del ML en seguridad, basado en el reconocimiento de desviaciones del patrón normal (*baselining*).

- **Algoritmos de *Clustering*:**
  - **Técnicas:** Algoritmos como **K-means** o **DBSCAN** se utilizan para agrupar patrones de tráfico de red o comportamientos de usuarios.

- **Propósito:** Identificar aquellos patrones que se desvían significativamente de los clústeres normales, lo que puede indicar una actividad maliciosa. Un ejemplo de esto se visualizó en la práctica cuando los valores atípicos (*outliers*) quedaban fuera de las agrupaciones esperadas en un gráfico de dispersión.
- **Análisis de Series Temporales:**
  - **Modelos:** Se emplean modelos como **ARIMA** o **LSTM** (los cuales se verán más en profundidad más adelante).
  - **Propósito:** Analizan secuencias de eventos en el tiempo para detectar desviaciones respecto a los patrones normales. Son muy útiles en la **detección de ataques DDOS** o anomalías en el acceso a sistemas críticos.

## 2.2. Predicción de Amenazas y Vulnerabilidades

El Machine Learning permite pasar de la detección reactiva a la prevención proactiva mediante la predicción de futuros incidentes.

- **Modelos Predictivos:**
  - **Algoritmos:** Se pueden utilizar algoritmos como la **regresión logística**, **árboles de decisión** y **Random Forest**.
  - **Propósito:** Predecir la **probabilidad de incidentes de seguridad** a partir de indicadores de compromiso y datos históricos de vulnerabilidades conocidas.
- **Redes Neuronales Profundas (*Deep Neural Networks*):**
  - **Aplicación Clave:** Son especialmente útiles para el **análisis de malware**.
  - **Capacidad:** Estas redes pueden aprender características complejas de archivos y procesos para identificar **comportamientos maliciosos**, incluso en *malware* **polimórfico o metamórfico** que cambia continuamente su código para evadir la detección tradicional.

## 2.3. Automatización de Respuestas a Incidentes

El ML acelera la mitigación de las amenazas al integrar inteligencia en la respuesta operativa.

- **Sistemas de Recomendación:**
    - **Mecanismo:** Basados en técnicas de *Machine Learning*, estos sistemas pueden **sugerir acciones de mitigación** o parches de seguridad relevantes para fallos de seguridad ya identificados y controlados.
    - **Beneficio:** Esto acelera significativamente la respuesta a incidentes y reduce el tiempo de permanencia de los atacantes.
  - **Aprendizaje por Refuerzo (*Reinforcement Learning*):**
    - **Técnica:** Esta técnica es muy buena para el **esfuerzo del aprendizaje**. Permite a los sistemas de seguridad **aprender y adaptar sus estrategias de defensa en tiempo real**, tomando decisiones sobre la mejor forma de responder a las amenazas basándose en el resultado de las acciones anteriores.
- 

### 3. Desafíos Críticos en la Implementación del Machine Learning

A pesar de sus grandes beneficios, el ML en ciberseguridad enfrenta desafíos considerables que deben ser gestionados cuidadosamente para un despliegue efectivo.

- **Evolución de Amenazas:** La naturaleza dinámica de las ciberamenazas, donde los atacantes constantemente desarrollan **nuevas técnicas**, requiere modelos de *Machine Learning* que sean capaces de **adaptarse y aprender continuamente** (*continuous learning*).
- **Falsos Positivos y Falsos Negativos:** La precisión en las detecciones es crítica:
  - **Falsos Positivos:** Demasiados pueden llevar a la **fatiga de alertas** en los equipos de seguridad.
  - **Falsos Negativos:** Representan **amenazas no detectadas** con consecuencias graves.
  - **Mitigación:** La selección de las características (*feature engineering*) y el ajuste fino (*fine tuning*) de los modelos son esenciales para minimizar estos errores.

- **Privacidad y Ética:** El uso de **datos sensibles** (ej., tráfico de red de usuarios) para entrenar modelos de *Machine Learning* plantea serias preocupaciones sobre la privacidad y la ética. Esto requiere buscar un **equilibrio entre la seguridad y el respeto por la privacidad** de los usuarios.
- 

## Conclusiones

La integración del **Machine Learning** en la ciberseguridad es fundamental para mejorar la detección de amenazas y la protección de sistemas. Al aplicar algoritmos de ML, se pueden **identificar patrones de comportamiento y anomalías** en los datos de manera altamente eficiente, fortaleciendo las defensas contra *malware* y ciberamenazas automatizadas.

Sin embargo, este enfoque requiere una **gestión muy cuidadosa de los riesgos** asociados con posibles sesgos en los modelos y la **evolución constante** de los mismos para mantenerse al día con las tácticas de los adversarios en un entorno que está siempre en cambio constante.

# Redes Neuronales y Mecanismos de Entrenamiento en Machine Learning

En esta sesión, realizaremos un repaso superficial pero crucial a los conceptos fundamentales de las **Redes Neuronales**. Es vital comprender estos puntos clave, como las funciones de activación, el Descenso de Gradiente (*Gradient Descent*) y la Retropropagación (*Backpropagation*), ya que son la base para el entrenamiento y la aplicación de modelos de ML en tareas de ciberseguridad, como la detección de anomalías o *malware*.

---

## 1. Estructura y Componentes de las Redes Neuronales

Las redes neuronales se inspiran en el funcionamiento del cerebro humano y se construyen a partir de unidades de procesamiento básicas llamadas **neuronas**.

### 1.1. Neuronas y Conexiones

- **Proceso Básico:** Cada neurona recibe múltiples **entradas**. Estas entradas se combinan basándose en sus **pesos** y se les añade un término de **sesgo** (*bias*). Esta combinación ajusta el nivel de activación de la neurona independientemente de las entradas. Finalmente, se aplica una **función de activación** para producir una salida.
- **Peso (*Weight*):** Cada conexión entre las neuronas tiene un peso que determina la **fuerza de la influencia** de una neurona sobre la otra.
- **Sesgo (*Bias*):** Es un parámetro adicional en cada neurona que permite **ajustar la salida de la función de activación**. Actúa como un umbral; por ejemplo, si una neurona debe activarse solo si la suma ponderada de sus entradas supera un cierto umbral (ej., 4), el sesgo ajusta este umbral.

### 1.2. Capas de la Red

Las neuronas se organizan en capas para procesar los datos de entrada y realizar tareas complejas como el reconocimiento de patrones, la clasificación y la predicción.

- **Capa de Entrada (*Input Layer*):** Es la capa que recibe directamente los **datos brutos**. Cada neurona en esta capa representa una **característica** de los datos de entrada.
- **Capas Ocultas (*Hidden Layers*):** Estas capas **transforman los datos** mediante pesos y funciones de activación.
  - **Función:** Permiten a la red aprender **jerarquías de características**. Por ejemplo, en la detección de objetos en imágenes, las primeras capas pueden identificar bordes, las capas intermedias pueden identificar formas simples a partir de esos bordes, y las capas más profundas pueden identificar objetos completos a partir de esas formas.
  - **Profundidad:** El número de capas ocultas contribuye a la **profundidad de la red** y, por ende, a su capacidad para aprender patrones complejos.
- **Capa de Salida (*Output Layer*):** Aquí es donde se produce el **resultado final del modelo**, que puede ser una clasificación o una predicción.

### 1.3. Funciones de Activación

La **función de activación** es crítica porque permite a las redes neuronales realizar tareas más allá de lo que los modelos lineales pueden hacer, habilitando el **aprendizaje de patrones complejos y sutiles** en grandes volúmenes de datos.

- **Propósito:** Transforma la salida combinada de una neurona para decidir si esta debe ser "activada" o no.
- **Ejemplos Comunes y Aplicación:**
  - **ReLU (Rectified Linear Unit):** Es la función de activación **más utilizada en las redes neuronales profundas** debido a su simplicidad computacional y porque reduce el problema del **desvanecimiento del gradiente** (un término técnico asociado al aprendizaje). ReLU transforma todos los valores negativos en cero, manteniendo los

positivos inalterados, y es ideal para **capas ocultas** en la mayoría de las redes.

- **Sigmoide ( $\sigma(x)$ ):** Comprime su entrada entre un **rango de 0 y 1**. Esto la hace muy útil para la **capa de salida en problemas de clasificación binaria**, donde la salida se interpreta como una probabilidad (ej., clasificar una imagen como "gato" o "no gato"). Sin embargo, no se recomienda para capas ocultas debido a problemas como el desvanecimiento del gradiente.

**ReLU (Rectified Linear Unit):**

$$f(x) = \max(0, x)$$

**Sigmoide:**

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- **Otras Funciones y Selección:** **Softmax** es ideal para la capa de salida en la **clasificación multiclase**, distribuyendo probabilidades entre varias clases. La elección de la mejor función depende del tipo de problema (regresión, clasificación), la arquitectura de la red y el comportamiento específico de los datos. Por ello, la **experimentación y el ajuste son cruciales** para identificar la mejor opción.

---

## 2. Mecanismos de Entrenamiento y Optimización

El entrenamiento de una red neuronal se basa en un ciclo iterativo de dos algoritmos fundamentales: la **Retropropagación** y el **Descenso de Gradiente**.

### 2.1. Descenso de Gradiente (*Gradient Descent*)

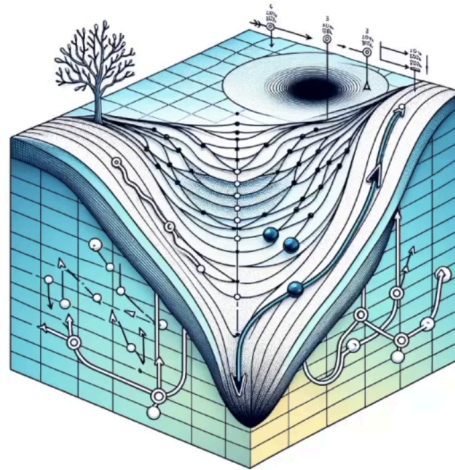
El Descenso de Gradiente es un **método de optimización** utilizado para actualizar los pesos y los sesgos en la dirección que **minimiza la función de pérdida** (*loss function*).

- **Analogía:** Imagina que estás en una montaña con niebla y quieres llegar al punto más bajo del valle (la función de pérdida mínima). El Descenso de Gradiente te indica en qué dirección debes dar el siguiente paso para



descender lo más rápido posible, basándose en la pendiente del terreno a tus pies.

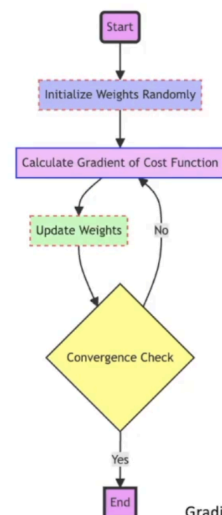
- **Principio Matemático:** La pendiente en cada punto se calcula tomando la **derivada** o el **gradiente de la función de pérdida** con respecto a los parámetros del modelo. Para minimizar la pérdida, es necesario mover los parámetros en la **dirección opuesta al gradiente**, que es la dirección en la



- **Flujo del Proceso:** El Descenso de Gradiente es un proceso iterativo que se repite hasta que el modelo converge a un mínimo:

#### Steps of Gradient Descent:

- **Initialization:** Starts with random values for model parameters.
- **Gradient Calculation:** Evaluates the loss function's gradient with respect to each parameter.
- **Parameter Update:** Adjusts parameters in the opposite direction of the gradient, subtracting a piece of the gradient from the current parameters, determined by the learning rate.
- **Repetition:** Repeats steps until the model converges to a minimum, or the changes in the loss function become insignificant.



Gradient Descent flow  
Picture source: own creation

**Inicialización:** Comienza asignando valores aleatorios a los parámetros del modelo (pesos y sesgos).

**Cálculo del Gradiente:** Se evalúa el gradiente de la función de coste con respecto a cada parámetro, indicando la dirección de mayor aumento de la pérdida.

**Actualización de Parámetros:** Los parámetros se ajustan en la dirección opuesta al gradiente. Se resta un fragmento del gradiente de los parámetros actuales, siendo el tamaño de ese fragmento (la magnitud del paso) determinado por la tasa de aprendizaje.

**Repetición (Convergence Check):** Se repiten los pasos hasta que el modelo converge a un mínimo o los cambios en la función de pérdida se vuelven insignificantes.

- **Hiperparámetros (Tasa de Aprendizaje):** La **tasa de aprendizaje** es un hiperparámetro crucial que determina **cuánto se ajustan los pesos y sesgos con cada actualización**.
  - Una tasa **muy alta** puede llevar a un aprendizaje inestable, sobrepasando el mínimo.
  - Una tasa **muy baja** puede hacer que el entrenamiento sea demasiado lento.
- **Desafíos:** En funciones de pérdida muy complejas, el algoritmo puede quedar atrapado en **mínimos locales y puntos de silla** (*saddle points*), donde el gradiente es cero, impidiendo alcanzar el mínimo global. Para afrontar esto, existen variantes como el Descenso del Gradiente Estocástico (SGD) y el Descenso del Gradiente con *momentum*.

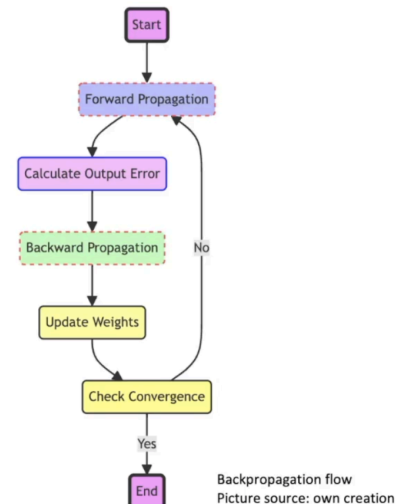
## 2.2. Retropropagación (*Backpropagation*)

La **Retropropagación** o *Backpropagation* es el **algoritmo esencial y central para el entrenamiento** de las redes neuronales, que permite ajustar los pesos y sesgos en todas las capas.

- **Propósito:** Minimizar la función de pérdida calculando el gradiente de la pérdida con respecto a cada peso y sesgo en la red.
- **Mecanismo:** Utiliza la **regla de la cadena del cálculo diferencial** y el cálculo de derivadas parciales para propagar el error hacia atrás, desde la capa de salida hasta la capa de entrada. Esto determina cuánto contribuyó cada parámetro al error final.

### The Core of Neural Network Training

**Backpropagation Essentials:** Backpropagation, or backward propagation of errors, is a fundamental algorithm in deep neural network training. It adjusts the weights and biases across all layers of the network to minimize the loss function by calculating the gradient of the loss with respect to each parameter (weight and bias) in the network, using partial derivatives and the chain rule of calculus.



- **Flujo del Proceso (Ciclo Iterativo):**
  1. **Propagación hacia Adelante (*Forward Propagation*):** La red toma los datos de entrada y los propaga a través de todas las capas para realizar una **predicción**.
  2. **Cálculo del Error (*Calculate Output Error*):** Se compara la predicción de la red con el valor real (la etiqueta). La diferencia cuantifica el error o la "pérdida".
  3. **Propagación hacia Atrás (*Backward Propagation*):** El error calculado se propaga hacia atrás a través de las capas. Se calcula el gradiente del error.
  4. **Actualización de Pesos (*Update Weights*):** Se utiliza el gradiente calculado (mediante el Descenso de Gradiente) para ajustar los pesos y sesgos, moviendo el modelo en la dirección que reduce el error.
  5. **Verificación de Convergencia (*Check Convergence*):** Se comprueba si el modelo ha alcanzado un rendimiento aceptable. Si no, el ciclo se repite.

En resumen, la Retropropagación es el mecanismo que le dice a la red neuronal **cómo corregir sus errores** en cada paso de entrenamiento para mejorar sus predicciones de forma gradual.

---

## Conclusiones

Es vital entender los conceptos básicos de las redes neuronales, incluyendo la arquitectura (neuronas, capas, pesos y sesgos), las funciones de activación, y los mecanismos de entrenamiento como la **Retropropagación** y el **Descenso de Gradiente**.

Aunque no es necesario un entendimiento técnico profundo a nivel matemático (ese no es el objetivo del máster), es fundamental comprender **cómo funcionan** y **cuáles son los hiperparámetros que se pueden ajustar**. Estos fundamentos nos permiten diseñar y entrenar modelos adecuados para la detección de intrusiones, el análisis de comportamiento de usuarios y la identificación de patrones de actividad maliciosa, implementando eficazmente técnicas de *Machine Learning* en aplicaciones de ciberseguridad.

# Detección de Malware mediante Machine

## Learning: Aplicación Práctica con Random Forest y XGBoost

En esta sesión práctica, abordaremos el proceso completo para crear y evaluar un proyecto de detección de *malware* utilizando algoritmos avanzados de *Machine Learning*: **Random Forest** y **XGBoost**. Analizaremos paso a paso el ciclo de vida del modelo, desde la carga de un *dataset* desbalanceado hasta la evaluación de su precisión con datos conocidos y datos nuevos.

---

### 1. Descripción y Preparación del Dataset

El conjunto de datos proporcionado es la base de nuestro entrenamiento, conteniendo información crucial para clasificar archivos ejecutables.

#### A. Características del Dataset

El *dataset* utilizado contiene **características (features) extraídas de archivos ejecutables de Windows**, los cuales han sido clasificados como maliciosos o no maliciosos. Estas características son de naturaleza **híbrida**, combinando representaciones en hexadecimal, binario y llamadas a las librerías dinámicas (DLLs).

- Dataset:

| Aspect                | Description                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Data Type             | Features extracted from Windows executable files, classified as malicious or non-malicious.         |
| Total Samples         | 373                                                                                                 |
| Malicious Samples     | 301                                                                                                 |
| Non-Malicious Samples | 72                                                                                                  |
| Total Features        | 531                                                                                                 |
| Relevant Features     | Some features may have little relevance, requiring feature selection to optimize model performance. |

## B. Objetivo del Proyecto

Nuestro objetivo es crear y evaluar un modelo de clasificación de *Machine Learning* capaz de distinguir entre archivos ejecutables **maliciosos y no maliciosos** utilizando las 531 *features* proporcionadas. Dado que la distribución es **desbalanceada** (301 maliciosas vs. 72 no maliciosas), este aspecto debe ser considerado, ya que podría afectar la capacidad de generalización de los modelos.

---

## 2. Flujo de Trabajo de ML: Random Forest (Dataset Original)

Utilizaremos **Jupyter Notebook** con **Pandas** y **Scikit-Learn** para ejecutar el flujo de trabajo.

### A. Importación de Librerías y Algoritmos

El primer paso es importar las herramientas necesarias de **Scikit-Learn** para el preprocesamiento, el modelado y la evaluación.

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score,
classification_report
```

**train\_test\_split**: Función esencial para dividir el conjunto de datos en conjuntos de entrenamiento y prueba, evaluando el rendimiento en datos no vistos.

**StandardScaler**: Clase utilizada para **estandarizar** las *features* (eliminando la media y escalando a varianza unitaria), un proceso crítico ya que muchos algoritmos asumen que todas las *features* tienen la misma escala.

**RandomForestClassifier**: El algoritmo de aprendizaje supervisado que utiliza un conjunto de **árboles de decisión** (un bosque) para la clasificación. Es conocido por su resistencia al sobreajuste y su capacidad para manejar datos complejos.

**accuracy\_score y classification\_report**: Métricas utilizadas para calcular la precisión y generar un informe detallado (precisión, sensibilidad, etc.) del rendimiento del modelo.

## B. Preprocesamiento de Datos

Una vez importadas las herramientas, se carga y se prepara el *dataset* (`uci_malware_detection.csv`).

**Carga del Dataset**: Se carga el fichero en un DataFrame de Pandas:

```
data = pd.read_csv('uci_malware_detection.csv')
```

*(Consejo práctico: Se puede usar `!ls` en Jupyter para confirmar el nombre del archivo en la carpeta de trabajo).*

**Manejo de Etiquetas Categóricas (Label Encoding)**: Se convierten las etiquetas de texto a valores numéricos, un requisito para la mayoría de los algoritmos de ML.

```
# Convert 'Label' to numeric values: non-malicious -> 0,  
malicious -> 1
```

```
data['Label'] = data['Label'].map({'non-malicious': 0,  
'malicious': 1})
```

- **Propósito:** Se accede a la columna `Label` y se utiliza la función `map` de Pandas para asignar `0` a 'non-malicious' y `1` a 'malicious', reasignando el resultado a la misma columna.

### Separación de Características (X) y Etiquetas (y):

```
# Splitting the dataset into features and labels
```

```
X = data.drop('Label', axis=1) # X: Contiene todas las  
features (se quita 'Label' a lo largo del eje 1)
```

```
y = data['Label'] # y: Contiene solo las  
etiquetas (la columna 'Label')
```

- **Propósito:** Se separan las **características (X)** que se usarán para entrenar el modelo de las **etiquetas (y)** que son el resultado esperado.

### Normalización de Datos (*StandardScaler*):

```
# Data normalization
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

- **`scaler = StandardScaler()`:** Se crea el objeto de estandarización.
- **`X_scaled = scaler.fit_transform(X)`:** El método `fit_transform` calcula la media y la desviación estándar de cada *feature* en X y luego aplica la transformación, asignando las *features* escaladas a X\_scaled.

### División en Conjuntos de Entrenamiento y Prueba:

```
# Splitting data into training and test sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled,  
y, test_size=0.2, random_state=42)
```



- **Propósito:** Divide el conjunto de datos escalado. Se utiliza `test_size=0.2` para reservar el **20% de los datos para el conjunto de prueba** y el 80% restante para el entrenamiento.
- `random_state=42`: Es una **semilla aleatoria** que garantiza la reproducibilidad de la división.

## C. Entrenamiento y Evaluación del Modelo (Random Forest)

### Selección y Entrenamiento:

```
# Model selection and training
```

```
model = RandomForestClassifier(n_estimators=100,
random_state=42)
```

```
model.fit(X_train, y_train)
```

- `n_estimators=100`: Este es un **hiperparámetro** que indica que se crearán **100 árboles de decisión** en el bosque aleatorio.
- `model.fit(X_train, y_train)`: La fase clásica de entrenamiento, donde el modelo ajusta sus parámetros internos para aprender de las *features* de entrenamiento (`X_train`) y sus etiquetas (`y_train`).

### Evaluación:

```
# Model evaluation
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
print("Classification Report:\n", report)
```

- `model.predict(X_test)`: Utiliza el modelo entrenado para hacer predicciones (`y_pred`) sobre las *features* de prueba.
- `accuracy_score(y_test, y_pred)`: Calcula la **precisión** del modelo (proporción de muestras clasificadas correctamente).

**Resultado del Random Forest (Datos Originales):** La precisión obtenida es muy alta, cercana al **98%**.

#### D. Prueba de Generalización con Datos Nuevos

Para evaluar la capacidad de **generalización** del modelo, se utiliza un nuevo *dataset* (`uci_malware_detection_new_data.csv`) que el modelo nunca ha visto.

##### Carga y Preprocesamiento del Nuevo Dataset:

```
new_data = pd.read_csv('uci_malware_detection_new_data.csv')
```

```
# Preprocessing
```

```
X_new = new_data.drop('Label', axis=1)
```

```
y_new = new_data['Label']
```

```
# IMPORTANTE: Solo se usa transform, NO fit_transform, ya que  
se reutiliza la media/std del set original
```

```
X_new_scaled = scaler.transform(X_new)
```

- **Clave de la Normalización:** Se utiliza el método `scaler.transform(X_new)` y **no** `fit_transform`. Esto garantiza que los nuevos datos se normalicen utilizando la media y desviación

estándar calculadas durante el entrenamiento del *dataset* original (X), replicando el proceso de producción.

### **Predicción y Evaluación con Datos Nuevos:**

```
y_pred_new = model.predict(X_new_scaled)

accuracy_new = accuracy_score(y_new, y_pred_new)

report_new = classification_report(y_new, y_pred_new)

print(f'Accuracy on New Data: {accuracy_new}')

print("Classification Report on New Data:\n", report_new)
```

**Resultado del Random Forest (Datos Nuevos):** La precisión fue muy baja, de solo el **19%** (0.19...).

**Conclusión:** El bajo rendimiento sugiere que el modelo **no está generalizando bien** con datos nuevos. Esto puede deberse a **sobreajuste** (*overfitting*) al *dataset* original. Por lo tanto, se justifica probar un algoritmo diferente.

### **Bloque de código entero usando Random Forests con el dataset original incluyendo el entrenamiento:**

```
data = pd.read_csv('uci_malware_detection.csv')

# Preprocessing: Handling categorical labels
# Convert 'Label' to numeric values: non-malicious -> 0, malicious -> 1
data['Label'] = data['Label'].map({'non-malicious': 0, 'malicious': 1})
```

```
# Splitting the dataset into features and labels
X = data.drop('Label', axis=1)
y = data['Label']

# Data normalization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Splitting data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

# Model selection and training
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Model evaluation
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

**Bloque de código entero usando Random Forests con el dataset nuevo sin incluir el entrenamiento (el modelo ya está entrenado):**

```
# Carga de datos
new_data = pd.read_csv('uci_malware_detection_new_data.csv')
X_new = new_data.drop('Label', axis=1)
y_new = new_data['Label']

# Preprocesamiento
X_new_scaled = scaler.transform(X_new)

# Predicción en el nuevo dataset
y_pred = model.predict(X_new_scaled)
```

```
# Evaluación
accuracy = accuracy_score(y_new, y_pred)
new_report = classification_report(y_new, y_pred) # Nueva línea para
reporte

print(f'Accuracy on New Data: {accuracy}')
print("Classification Report on New Data:\n", new_report)
```

---

### *3. Flujo de Trabajo de ML: XGBoost*

Se prueba el algoritmo **XGBoost** (*eXtreme Gradient Boosting*) para buscar una mejor capacidad de generalización.

#### **A. Instalación e Inicialización**

**Instalación:**

```
!pip install xgboost
```

**Importación:**

```
from xgboost import XGBClassifier
```

#### **B. Entrenamiento y Evaluación (Dataset Original)**

Se repiten los pasos de preprocesamiento, pero se cambia la instanciación del modelo:

```
data = pd.read_csv('uci_malware_detection.csv')

# Preprocessing: Handling categorical labels
# Convert 'Label' to numeric values: non-malicious -> 0, malicious
-> 1
data['Label'] = data['Label'].map({'non-malicious': 0, 'malicious':
1})
```

```

# Splitting the dataset into features and labels
X = data.drop('Label', axis=1)
y = data['Label']

# Data normalization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Splitting data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

# Instanciación y Entrenamiento del Modelo XGBoost
model = XGBClassifier(use_label_encoder=False,
eval_metric='logloss')
model.fit(X_train, y_train)

# Model evaluation
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:\n", report)

```

- **use\_label\_encoder=False**: Opción de versión para evitar *warnings* o problemas específicos.
- **eval\_metric='logloss'**: XGBoost requiere una métrica de evaluación durante el entrenamiento; *logloss* es una elección común para la clasificación binaria y guía el proceso de optimización.

**Resultado del XGBoost (Datos Originales):** La precisión fue de **≈98.6%**, lo que indica un rendimiento ligeramente superior al Random Forest con datos conocidos.

### C. Prueba de Generalización con Datos Nuevos (XGBoost)

Se repite la prueba clave de generalización, utilizando el modelo XGBoost entrenado.

```
new_data = pd.read_csv('uci_malware_detection_new_data.csv')

X_new = new_data.drop('Label', axis=1)
y_new = new_data['Label']

scaler = StandardScaler()
scaler.fit(X)

X_new_scaled = scaler.transform(X_new)

# Predicción en el nuevo dataset (X_new_scaled)
y_pred_new = model.predict(X_new_scaled)

# Evaluación
accuracy_new = accuracy_score(y_new, y_pred_new)
report_new = classification_report(y_new, y_pred_new)

print("Accuracy on the new dataset:", accuracy_new)
print("Classification Report on the new dataset:\n", report_new)
```

**Resultado del XGBoost (Datos Nuevos):** La precisión fue de  $\approx 61\%$  (0.61...).

**Conclusión Final:** Aunque la precisión de 61% aún no es óptima, representa una **mejora significativa** respecto al 19% obtenido con *Random Forest*. Esto demuestra que **XGBoost** exhibe una **mejor capacidad de generalización** para este problema de detección de *malware*, lo cual subraya la importancia de la selección del algoritmo adecuado en *Machine Learning* aplicado a ciberseguridad.

# Herramientas Avanzadas para Machine Learning y

## Deep Learning en Ciberseguridad

En esta sesión, exploraremos varias herramientas críticas utilizadas en el desarrollo de aplicaciones de **Machine Learning (ML)** y **Deep Learning (DL)**. Si bien su complejidad es alta, las utilizaremos frecuentemente en el módulo, por lo que es esencial familiarizarse con sus conceptos y funcionamiento básico.

---

### *1. TensorFlow: Fundamento del Deep Learning*

**TensorFlow** es una biblioteca de *software* de código abierto creada por el equipo de Google Brain. Es una de las bibliotecas más populares para la implementación de algoritmos de aprendizaje automático, especialmente aquellos relacionados con **redes neuronales profundas**.

#### **1.1. Capacidades Clave**

- **Eficiencia en Cálculos:** Permite construir y entrenar modelos con una alta eficiencia en términos de cálculos matemáticos, gracias a su capacidad de ejecución en **múltiples CPU o GPUs**.
- **Autodiferenciación:** Su capacidad de autodiferenciación calcula automáticamente las derivadas de las funciones definidas por el usuario, lo que simplifica enormemente la implementación de algoritmos complejos, como el Descenso de Gradiente (*Gradient Descent*).

#### **1.2. Tensores (Estructuras de Datos Multidimensionales)**

TensorFlow utiliza una estructura de datos principal llamada **tensor**, que es un conjunto organizado de datos que puede tener múltiples dimensiones.

- **Rango (*Rank*):** Se refiere al **número de dimensiones** que tiene el tensor (ej., un vector tiene rango 1, una matriz tiene rango 2).



- **Forma (*Shape*):** Describe el **tamaño de cada dimensión** del tensor (ej., una matriz 3x3 tiene forma `(3, 3)`).
- **Tipo de Datos (*Dtype*):** Especifica el tipo de elementos que contiene el tensor, como enteros (`int32`), flotantes (`float32`) o cadenas.

### 1.3. Procedimiento Práctico: Creación e Inspección de Tensores

Para aquellos que no están familiarizados con el *Machine Learning*, la palabra **tensor** puede sonar abstracta. Sin embargo, un tensor no es más que un contenedor de datos diseñado para ser manipulado eficientemente en cálculos matriciales. Es esencialmente una generalización de conceptos que ya conocemos: una lista de números es un tensor de rango 1, y una matriz (o una tabla) es un tensor de rango 2.

El siguiente procedimiento demuestra cómo TensorFlow crea estos contenedores y cómo podemos inspeccionar su estructura, lo cual es el primer paso para preparar datos de seguridad para una red neuronal.

#### A. Creación e Inspección del Tensor de Rango 1 (Vector)

El **Rango** de un tensor se refiere al número de dimensiones que posee. Un tensor de rango 1 es el equivalente a un **vector** o una lista simple, una estructura de una sola dimensión.

Asumiendo que TensorFlow está instalado, el siguiente código demuestra la creación de tensores de diferente rango y la extracción de su información estructural.

```
import tensorflow as tf

# Tensor de Rango 1 (Vector unidimensional)
tensor_rank1 = tf.constant([1, 2, 3, 4, 5])

print("Rank of the tensor:", tf.rank(tensor_rank1))
print("Shape of the tensor:", tf.shape(tensor_rank1))
print("Data type of the tensor:", tensor_rank1.dtype)
```

## Output de la terminal tras ejecutar el Tensor de rango 1:

```
Rank of the tensor: tf.Tensor(1, shape=(), dtype=int32)
Shape of the tensor: tf.Tensor([5], shape=(1,), dtype=int32)
Data type of the tensor: <dtype: 'int32'>
```

El comando `tf.constant` crea el tensor a partir de la lista de números. Luego, utilizamos funciones de TensorFlow para inspeccionar su estructura:

1. **Análisis del Rango (Número de Dimensiones):** Al ejecutar `tf.rank(tensor_rank1)`, el *output* de la terminal nos muestra el valor **1**. Esto confirma que el tensor tiene **una sola dimensión**. Es importante destacar que el **rango** de un tensor se refiere únicamente al número de dimensiones que posee.
2. **Análisis de la Forma (Shape):** La ejecución de `tf.shape(tensor_rank1)` nos proporciona el valor **[5]**. Este resultado, llamado la **Forma** o *Shape*, nos dice que a lo largo de esa única dimensión, el tensor contiene exactamente **5 elementos** (los números del 1 al 5). La forma siempre describe el tamaño de cada dimensión.
3. **Análisis del Tipo de Dato (Dtype):** Finalmente, al acceder a `tensor_rank1.dtype`, observamos el *output* `<dtype: 'int32'>`. Esto confirma que todos los elementos contenidos en el tensor son **números enteros** (*int*) y que cada uno ocupa **32 bits** de memoria, algo crucial para la eficiencia en el procesamiento.

## B. Creación e Inspección del Tensor de Rango 2 (Matriz)

Un tensor de rango 2 es el equivalente a una **matriz** o una tabla bidimensional, con filas y columnas. Se representa como una lista anidada (una lista que contiene otras listas).

```
# Tensor de Rango 2 (Matriz bidimensional)
tensor_rank2 = tf.constant([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
```

```
        [7.0, 8.0, 9.0]
    ])

print("\nRank of the tensor:", tf.rank(tensor_rank2))
print("Shape of the tensor:", tf.shape(tensor_rank2))
print("Data type of the tensor:", tensor_rank2.dtype)
```

### Output de la terminal tras ejecutar el Tensor de rango 2:

```
Rank of the tensor: tf.Tensor(2, shape=(), dtype=int32)
Shape of the tensor: tf.Tensor([3 3], shape=(2,), dtype=int32)
Data type of the tensor: <dtype: 'float32'>
```

1. **Análisis del Rango:** Al ejecutar `tf.rank(tensor_rank2)`, el *output* de la terminal es `2`. Esto verifica que el tensor es bidimensional, es decir, una **matriz** con filas y columnas.
2. **Análisis de la Forma (Shape):** El comando `tf.shape(tensor_rank2)` devuelve `[3 3]`. La **Forma** nos dice, en este caso, que el tensor tiene **3 filas** y **3 columnas**. Comprender la forma es vital para saber si los datos de entrada coinciden con las expectativas del modelo de *Deep Learning*.
3. **Análisis del Tipo de Dato (Dtype):** El resultado `<dtype: 'float32'>` indica que los elementos son de **punto flotante (float)** de **32 bits**. TensorFlow infiere esto automáticamente debido a que los números fueron definidos con un punto decimal (ej., `1.0`). El uso de valores de punto flotante es habitual en el *Deep Learning*, ya que permiten la alta precisión necesaria para calcular los gradientes y actualizar los pesos del modelo.

Entender la forma en que TensorFlow organiza los datos en términos de **Rango**, **Forma** y **Tipo de Dato** es la base para estructurar correctamente las características de seguridad (como los *features* de *malware* o los *logs* de red) antes de pasarlos a Keras para el entrenamiento.

Este ejercicio práctico es crucial porque el ML y el *Deep Learning* trabajan exclusivamente con tensores. Comprender su rango, forma y tipo de dato es el

primer paso para estructurar correctamente los datos de seguridad antes de alimentar una red neuronal.

---

## 2. Keras: API de Alto Nivel para Redes Neuronales

**Keras** es una biblioteca de programación de **alto nivel** para redes neuronales en Python. Se ejecuta como interfaz sobre plataformas de aprendizaje profundo como **TensorFlow**, Theano o Microsoft Cognitive Toolkit, aunque su uso actual está intrínsecamente ligado a TensorFlow.

### 2.1. Ventajas y Arquitectura

- **API Simple y Consistente:** Minimiza la cantidad de código necesario para construir modelos de *Deep Learning*.
- **Diseño Modular:** Está diseñado con módulos configurables e independientes que pueden combinarse fácilmente.
- **Soporte para Múltiples *Backends*:** Permite a los usuarios elegir la plataforma de optimización subyacente (aunque TensorFlow es el *backend* dominante).

### 2.2. Componentes Clave de Keras

- **Modelos:** Proporciona dos tipos principales:
  - **Secuencial:** Para apilar capas de forma lineal (el más sencillo).
  - **API Funcional:** Para construir modelos más complejos con múltiples entradas/salidas o conexiones no secuenciales.
- **Capas (*Layers*):** Son los bloques de construcción básicos de las redes neuronales (ej., capas densamente conectadas, capas de convolución, recurrentes, *pooling* y normalización).
- **Optimizadores:** Algoritmos utilizados para ajustar los pesos del modelo durante el entrenamiento (ej., **SGD** -Descenso de Gradiente Estocástico-, **ADAM**).

- **Funciones de Pérdida (*Loss Functions*):** Métricas que el modelo intenta minimizar durante el entrenamiento (ej., entropía cruzada, *mean\_absolute\_error*).
- **Funciones de Activación:** Proporciona una gran variedad de funciones como **ReLU**, **Softmax** y **Sigmoide**.

### 2.3. Procedimiento Práctico: Regresión con Keras y TensorFlow

Este ejemplo combina NumPy, TensorFlow y Keras para entrenar una red que predice el precio de una casa basándose en su tamaño.

**Preparación de Datos (NumPy):** Se definen los datos de entrada (tamaño de la casa) y salida esperada (precio).

```
import numpy as np
# Ejemplo de datos: tamaño de la casa (m²) y precio (miles de USD)
house_size = np.array([50, 60, 70, 80, 90])
price = np.array([150, 180, 210, 240, 270])
```

**Normalización de Datos:** Se normalizan los datos para que todos los valores caigan dentro del rango **0–1**. Esto ayuda significativamente al entrenamiento del modelo, ya que facilita el manejo y la comprensión de la información por parte del algoritmo.

```
# Normalize input data
house_size_normalized = house_size / np.max(house_size)
price_normalized = price / np.max(price)
```

- **Propósito:** Al dividir cada valor por el máximo, el valor máximo se convierte en 1 y el mínimo se mantiene proporcionalmente cercano a 0.

**Definición de la Arquitectura del Modelo (Keras Sequential API):** Se construye el modelo apilando capas linealmente.

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=64, activation='relu',
input_shape=[1]),
    tf.keras.layers.Dense(units=64, activation='relu'),
    tf.keras.layers.Dense(units=1)
])
```

- **tf.keras.Sequential(...)**: Indica que las capas se apilan en secuencia.
- **tf.keras.layers.Dense(...)**: Crea una capa densamente conectada (*Full Connected*).
- **Capa 1: units=64, activation='relu', input\_shape=[1]**. Capa de entrada con 64 neuronas. **input\_shape=[1]** especifica que solo hay una característica de entrada (el tamaño de la casa).
- **Capa 2: units=64, activation='relu'**. Una capa oculta con 64 neuronas. El *input\_shape* se infiere automáticamente.
- **Capa 3 (Salida): units=1**. La capa de salida tiene una sola neurona para la predicción de un único valor (el precio). La **ausencia de función de activación** implica una simple regresión lineal, adecuada para predecir un valor sin restricciones.

**Compilación del Modelo:** Se especifica la función de pérdida y el optimizador.

```
# Compile the model with a loss function and an optimizer
model.compile(loss='mean_absolute_error', optimizer='adam')
```

- **loss='mean\_absolute\_error'**: Función de pérdida que el modelo intentará minimizar.
- **optimizer='adam'**: El algoritmo optimizador utilizado para ajustar los pesos durante el entrenamiento.

### Entrenamiento del Modelo:

```
# Train the model
model.fit(house_size_normalized, price_normalized, epochs=200,
verbose=False)
```

- **model.fit(...)**: Comienza el proceso de entrenamiento utilizando los datos normalizados.
- **epochs=200**: El modelo verá todo el conjunto de datos 200 veces.
- **verbose=False**: Suprime la salida detallada del entrenamiento en la terminal.

**Predicción y Desnormalización:** Se predice el precio de una casa de 100 m2 y se convierte el resultado normalizado al precio real.

```
# Predict the price of a house with 100 square meters
(normalized)
predicted_price_normalized = model.predict(np.array([[100 /
np.max(house_size)]]))

# Denormalize the predictions
predicted_price = predicted_price_normalized * np.max(price)

print(predicted_price)
```

**Resultado de Ejemplo:** El resultado, `[[296.39383]]` (miles de dólares), encaja en la progresión de los datos de entrenamiento.

## Bloque de código entero del ejemplo de las casas con TensorFlow y Keras:

```
# Example data: house size (in square meters) and price in
thousands of dollars
house_size = np.array([50, 60, 70, 80, 90])
price = np.array([150, 180, 210, 240, 270])

# Normalize input data
house_size_normalized = house_size / np.max(house_size)
price_normalized = price / np.max(price)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=64, activation='relu',
input_shape=[1]),
    tf.keras.layers.Dense(units=64, activation='relu'),
    tf.keras.layers.Dense(units=1)
])

# Compile the model with a loss function and an optimizer
model.compile(loss='mean_absolute_error', optimizer='adam')

# Train the model
model.fit(house_size_normalized, price_normalized, epochs=200,
verbose=False)

# Predict the price of a house with 100 square meters
(normalized)
predicted_price_normalized = model.predict(np.array([[100 /
np.max(house_size)]]))

predicted_price = predicted_price_normalized * np.max(price)

print(predicted_price)
```

---



## 2.4. Adaptación a un Problema de Ciberseguridad

El siguiente bloque de código completo adapta el modelo de regresión anterior a un problema de ciberseguridad, utilizando dos características de entrada para predecir una **puntuación de seguridad** (*security score*).

El objetivo es demostrar lo sencillo que es adaptar estos modelos clásicos a contextos de ciberseguridad.

El mismo modelo se puede adaptar a un problema de ciberseguridad, como predecir una **puntuación de seguridad** (*security score*) basada en múltiples métricas de red.

- **Entradas (*Features*):** Se usan dos *features* de entrada: `login_attempts` (intentos de inicio de sesión) y `anomalous_traffic` (cantidad de tráfico anómalo).
- **Salida (*Target*):** `security_scores` (puntuaciones de seguridad entre 0.2 y 1.0).
- **Arquitectura Sencilla:** Se utiliza un modelo simple con una sola capa densa de salida, especificando `input_shape=[2]` ya que ahora hay dos *features* de entrada.

### A. Bloque de Código Completo y Definición de Datos:

```
import numpy as np
import tensorflow as tf

# Example data: number of login attempts and amount of anomalous
# network traffic
login_attempts = np.array([10, 20, 25, 30]) # Number of login
attempts
anomalous_traffic = np.array([100, 150, 200, 250, 300]) # Amount of
anomalous network traffic
security_scores = np.array([0.2, 0.4, 0.6, 0.8, 1.0]) # Example of
training targets (security scores)

# Normalize the input data
login_attempts_normalized = login_attempts / np.max(login_attempts)
```

```

anomalous_traffic_normalized = anomalous_traffic /
np.max(anomalous_traffic)
security_scores_normalized = security_scores /
np.max(security_scores)

# Define a simple model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[2]) # Specify 2
features as input
])

# Compile the model with a loss function and an optimizer
model.compile(loss='mean_absolute_error', optimizer='adam')

# Combine the normalized features as input data
X_train = np.column_stack((login_attempts_normalized,
anomalous_traffic_normalized))

# Define a simple model (specifying 2 features as input)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[2])
])

# Compile and Train
model.compile(loss='mean_absolute_error', optimizer='adam')
model.fit(X_train, security_scores_normalized, epochs=200,
verbose=False)

# Test data prediction (similar steps for normalization and
combination)
# ...

# Predict the security score
predicted_security_scores_normalized = model.predict(X_test)
predicted_security_scores = predicted_security_scores_normalized *
np.max(security_scores)

print(predicted_security_scores)

```

## **B. Análisis Detallado del Código:**

### **Definición de las Entradas y Salidas:**

- **login\_attempts**: Contiene el **número de intentos de inicio de sesión** para una red.
- **anomalous\_traffic**: Representa la **cantidad de tráfico de red anómalo** circulando.
- **security\_scores**: Se utilizan para representar los **objetivos de entrenamiento**. Son las puntuaciones de seguridad asociadas a los datos de entrada, que el modelo debe aprender a predecir.

### **Normalización:**

```
login_attempts_normalized = login_attempts /  
np.max(login_attempts)
```

- **Propósito**: La normalización es idéntica a la del ejemplo anterior (dividir por el valor máximo), asegurando que todas las *features* de entrada (**login\_attempts**, **anomalous\_traffic**) estén en el rango **0–1**. Esto es crucial para un entrenamiento más eficiente.

### **Definición del Modelo (Simplificado):**

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(units=1, input_shape=[2]) # Specify  
    2 features as input  
)
```

- **Arquitectura**: Se define un modelo extremadamente simple, con **una sola capa densa**.
- **input\_shape=[2]**: Se especifica que hay **dos características de entrada** (**login\_attempts** y **anomalous\_traffic**).

- **units=1**: La capa de salida tiene una única neurona para predecir la puntuación de seguridad.

### Combinación de Características para Entrenamiento:

```
X_train = np.column_stack((login_attempts_normalized,  
anomalous_traffic_normalized))
```

- **Propósito**: Se utiliza `np.column_stack` para combinar las dos *features* normalizadas en una única matriz de entrada (`X_train`), lista para el entrenamiento.

### Entrenamiento:

```
model.fit(X_train, security_scores_normalized, epochs=200,  
verbose=False)
```

- El modelo es entrenado, ajustando sus pesos para mapear las entradas combinadas (`X_train`) a las puntuaciones de seguridad normalizadas (`security_scores_normalized`).

### Predicción con Datos de Prueba (*Test Data*):

- Se definen **datos de prueba simulados** (`login_attempts_test`, `anomalous_traffic_test`).
- Los datos de prueba deben ser **normalizados** utilizando la misma lógica (dividir por el valor máximo del conjunto de entrenamiento original).
- Se utiliza `np.column_stack` para crear la matriz de prueba (`X_test`).
- **Predicción**: `predicted_security_scores_normalized = model.predict(X_test)`.

- **Desnormalización:** `predicted_security_scores = predicted_security_scores_normalized * np.max(security_scores).`

### C. Interpretación del Resultado

```
[[ 0.02390172]
 [-0.00125502]
 [-0.02641164]]
```

- **Estructura del Output:** La salida contiene la puntuación de seguridad predicha para el primer, segundo y tercer conjunto de datos de prueba, respectivamente.
  - **Interpretación de la Puntuación:** Las puntuaciones se encuentran en el rango (teórico) de -1 a 1.
    - **Valores más altos:** Indican **mayor seguridad**.
    - **Valores más bajos (incluyendo predicciones negativas):** Indican **menor seguridad**.
  - **Análisis del Ejemplo:** Los valores obtenidos (cerca de cero o ligeramente negativos) sugieren una **seguridad moderada o baja** para los datos de prueba introducidos, lo cual es coherente con el modelo simple utilizado.
  - **Conclusión Didáctica:** Este ejemplo ilustra la sencillez de adaptar los modelos de Keras a cualquier problema de ciberseguridad, aunque se debe ser cauteloso con la interpretación de las predicciones inesperadas (como los valores negativos), ya que el modelo es muy básico. Solo se requiere la correcta introducción y normalización de la información.
- 

## 3. Herramientas para el Procesamiento del Lenguaje Natural (NLP)

Las librerías de Procesamiento del Lenguaje Natural son cruciales en ciberseguridad para analizar textos relacionados con amenazas (ej., *logs*, *phishing*, *spam*).

### 3.1. NLTK (*Natural Language Toolkit*)

NLTK es la librería número uno para el trabajo con texto humano en Python.

- **Recursos Lingüísticos:** Ofrece acceso a una gran variedad de **corpora** y **recursos léxicos**. Un **corpus** se refiere a una colección estructurada de datos de texto (documentos, correos electrónicos, artículos) utilizados para entrenar, desarrollar y evaluar algoritmos de NLP.
- **Herramientas de Procesamiento:** Incluye herramientas para tareas básicas como la **tokenización** (división de texto), el **stemming** (reducir palabras a su raíz), el etiquetado de partes del discurso y el análisis sintáctico.
- **Accesibilidad:** Su diseño orientado a objetos y clara documentación hacen que el procesamiento de lenguaje natural sea muy **accesible** para principiantes, sin dejar de ser robusto para la investigación.
- **Uso:** Históricamente utilizado para el análisis de sentimiento, la clasificación de texto y la extracción de información. Es una opción eficiente si se busca tener control total y evitar modelos pesados (*Large Language Models* o LLM).

### 3.2. spaCy

spaCy es otra biblioteca de NLP muy popular, conocida por su **rapidez y eficiencia**.

- **Uso en Producción:** Está diseñada para casos de uso de **producción**, ofreciendo implementaciones preentrenadas de muchas tareas comunes de NLP (tokenización, etiquetado, etc.).
- **Escalabilidad:** Se centra en proporcionar modelos de procesamiento de lenguaje natural que sean consistentes, fácilmente **escalables** y extendidos.

Ambas librerías (NLTK y spaCy) se pueden integrar para analizar correos electrónicos en la búsqueda de **spam o phishing**, conectando así el análisis de texto con las capacidades de clasificación de TensorFlow/Keras.

---

## 4. Conclusión

**TensorFlow** y **Keras** son herramientas fundamentales para el desarrollo de aplicaciones de ciberseguridad, ya que facilitan la construcción de modelos de *Deep Learning* capaces de detectar y prevenir amenazas de manera eficiente.

Estas bibliotecas ofrecen la funcionalidad necesaria para el procesamiento de datos y la creación de redes neuronales, lo que es vital para sistemas de detección de intrusiones, análisis de *logs* e identificación de *malware*. Complementariamente, las bibliotecas de NLP como **NLTK** y **spaCy** son esenciales para analizar y comprender el texto relacionado con la ciberseguridad, como **registros de eventos**, **correos electrónicos** y **amenazas en redes sociales**.

# Arquitectura LSTM: Long Short-Term Memory en Ciberseguridad

En esta sesión, exploraremos la arquitectura de redes neuronales conocida como **Long Short-Term Memory (LSTM)**. Esta arquitectura es crucial en tareas de **procesamiento del lenguaje natural** y **series temporales**, campos que tienen una aplicación directa y relevante en el ámbito de la ciberseguridad.

El funcionamiento avanzado de una red LSTM se basa en su capacidad para **recordar información durante largos periodos de tiempo** y decidir, de manera selectiva, qué información debe ser mantenida, modificada o eliminada. Esto es vital en cualquier secuencia de datos donde la dependencia entre los elementos (como un evento de *log* que afecta a un evento posterior) es crítica.

---

## 1. Componentes Centrales de la Arquitectura LSTM

Una celda LSTM logra su memoria selectiva mediante una estructura interna compuesta por un estado central y un sistema de "puertas" (*Gates*) que regulan el flujo de información.

### 1.1. Estado de la Celda (*Cell State*)

- **Definición:** Es el componente central de la LSTM. Actúa como la **memoria de la red**, transportando información relevante a través de muchos pasos de tiempo.
- **Función:** Transporta la memoria a largo plazo a través de la secuencia, permitiendo que la red acceda a datos de eventos muy antiguos en la serie.

### 1.2. Puertas (*Gates*)

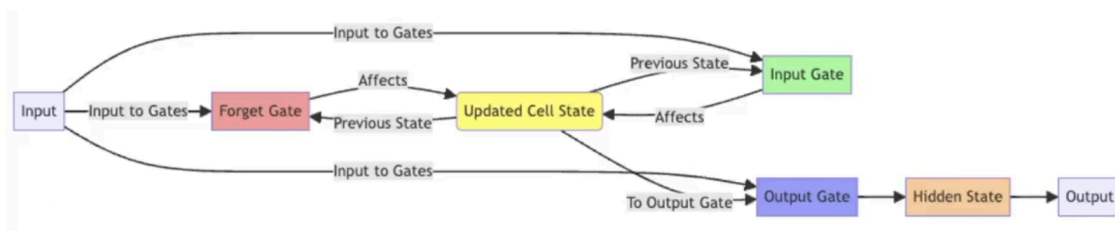
Las "puertas" son pequeñas redes neuronales dentro de la celda que controlan, mediante valores entre 0 y 1, cuánta información puede fluir a través de ellas.



- **Puerta del Olvido (*Forget Gate*):** Decide **qué información del estado de la celda se debe descartar** o mantener. Si la puerta genera un valor cercano a 0, la información se olvida; si es cercano a 1, se mantiene.
- **Puerta de Entrada (*Input Gate*):** Decide **qué información nueva** procedente de la entrada actual se va a **añadir al estado de la celda**.
- **Puerta de Salida (*Output Gate*):** Decide **qué parte del estado de la celda** se utilizará para calcular y **pasar a la salida** de la red.

## 2. Modo de Operación de la LSTM: Proceso Secuencial Continuo

La siguiente representación gráfica ilustra cómo la entrada, el estado previo y el *Cell State* interactúan a través de las puertas.



- **Input:** Representa la **entrada actual** que recibe la LSTM en un paso de tiempo determinado (ej., el último evento de tráfico de red).
- **Updated Cell State (Amarillo):** Este es el núcleo de la memoria central. Se **actualiza** continuamente con información nueva proveniente de la *Input Gate* y se limpia con la *Forget Gate*.
- **Forget Gate (Rojo):** Regula el impacto del **Previous State** (el estado de la celda en el paso de tiempo anterior) sobre el nuevo *Cell State*.
- **Input Gate (Verde):** Regula el impacto del **Input** (la entrada actual) sobre el nuevo *Cell State*.
- **Output Gate (Azul) y Hidden State (Naranja):** La *Output Gate* decide qué parte del **Updated Cell State** (memoria a largo plazo) es relevante para el **Hidden State** (estado oculto). Este *Hidden State* se calcula basándose en el estado actualizado de la celda y se utiliza para producir el **Output** final de la red.

## Explicación Detallada del Flujo Secuencial (Ejemplo de Recomendación)

Utilizaremos la analogía de una aplicación de **recomendación de música** para explicar el flujo de la LSTM como una única secuencia de eventos.

### Paso 1: Procesamiento de la Canción A (Inicio de la Secuencia)

1. **Input (Canción A):** El sistema recibe las características de la Canción A (género, ritmo).
2. **Forget Gate:** Dado que es la primera canción en la sesión, no hay memoria previa que olvidar. La puerta permite que el *Cell State* se reinicie o continúe.
3. **Input Gate:** Se activa, introduciendo las características de la Canción A, considerándolas relevantes para futuras recomendaciones.
4. **Updated Cell State:** El *Cell State* incorpora la información relevante de la Canción A.
5. **Output Gate:** La puerta utiliza el *Cell State* para la salida, pero decide **esperar** a tener más datos (más canciones) antes de generar una recomendación.

### Paso 2: Procesamiento de la Canción B (Continuación de la Secuencia)

1. **Input (Canción B):** El usuario escucha y el sistema recibe las características de la Canción B.
2. **Forget Gate:** La puerta evalúa el *Cell State* (que contiene la Canción A) y la nueva entrada (Canción B). Decide **mantener la información de la Canción A**, considerándola todavía relevante para las preferencias del usuario.
3. **Input Gate:** La puerta introduce las nuevas características de la **Canción B** en el *Cell State*.
4. **Updated Cell State:** El *Cell State* se **actualiza**, conteniendo ahora la memoria relevante de la Canción A y la Canción B.
5. **Output Gate:** Utilizando este *Cell State* con memoria a largo plazo (A + B), la puerta de salida decide generar una **recomendación** (ej., la Canción C).

### Ajuste Dinámico de la Memoria:

A medida que el usuario sigue escuchando más canciones, el sistema realiza un **ajuste dinámico** en cada paso temporal:

- Utiliza la **Puerta del Olvido** para decidir qué información anterior (memoria) es obsoleta y debe descartarse.
- Utiliza la **Puerta de Entrada** para incorporar la nueva información.

Este proceso secuencial y continuo permite a las LSTMs adaptarse a los patrones que escucha el usuario, siendo este el mecanismo clave que las hace tan efectivas en el análisis de series temporales y eventos de ciberseguridad.

---

### 3. Conclusiones y Relevancia en Ciberseguridad

Las LSTM son un tipo de **red neuronal recurrente (RNN)** que ha demostrado ser excepcionalmente efectiva en la **modelización de secuencia de datos**, lo que las hace muy relevantes en el ámbito de la ciberseguridad.

Su capacidad para manejar eficazmente datos secuenciales permite a las LSTM capturar **patrones temporales** y **relaciones de dependencia a largo plazo** en datos de seguridad, como por ejemplo:

- **Registro de Eventos (Logs):** Analizar la secuencia de eventos de un usuario o sistema a lo largo del tiempo.
- **Tráfico de Red:** Modelizar la secuencia de paquetes y la dependencia temporal entre las peticiones.
- **Secuencia de Comandos:** Analizar el orden y la estructura de los comandos ejecutados en un *host*.

En el mundo de la ciberseguridad, las LSTM se usan mucho para:

- **Detección de Intrusiones:** Identificando secuencias de eventos que difieren de la actividad normal.
- **Análisis de Anomalías de Comportamiento:** Detectando cuándo el comportamiento de un usuario o sistema se desvía del patrón aprendido.
- **Predicción de Ataques:** Modelando tendencias temporales en datos de amenazas.

# Procedimiento Práctico: Detección de Spam con Deep Learning y Arquitectura LSTM

En esta sesión, abordaremos la creación de un sistema de clasificación avanzado capaz de distinguir correos electrónicos (o mensajes de texto como SMS) que son potencialmente una amenaza (*spam*) de aquellos que son legítimos. Para este proyecto de clasificación binaria, utilizaremos **Deep Learning** con **TensorFlow/Keras** y una arquitectura de red basada en **Long Short-Term Memory (LSTM)**, complementada por librerías de Procesamiento del Lenguaje Natural (NLP) como **NLTK**.

El *dataset* para esta práctica, que consta de ficheros de entrenamiento y de prueba, se puede obtener en plataformas de datos públicas, lo que subraya la naturaleza accesible de estas fuentes para la investigación en ciberseguridad.

---

## Bloque de código entero del modelo que veremos ahora:

```
# Import required libraries
import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.layers import Input
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords') # Download stopwords from NLTK

# Load datasets with the correct encoding
train_data = pd.read_csv('SMS_train.csv', encoding='latin1') # Load training dataset
test_data = pd.read_csv('SMS_test.csv', encoding='latin1') # Load testing dataset

# Combine train and test datasets for text preprocessing
data = pd.concat([train_data, test_data], ignore_index=True)
```

```

# Text preprocessing function
def preprocess(text):
    # Convert to lowercase and tokenize
    tokens = nltk.word_tokenize(text.lower())
    # Remove stopwords
    stopwords = [word for word in tokens if word not in
stopwords.words('english')]
    return " ".join(stopwords)

# Apply preprocessing to SMS texts
data['processed_text'] = data['Message_body'].apply(preprocess)

# Tokenization and Padding
tokenizer = Tokenizer(num_words=5000) # Initialize tokenizer with a
max number of words
tokenizer.fit_on_texts(data['processed_text']) # Fit tokenizer on
texts
sequences = tokenizer.texts_to_sequences(data['processed_text']) #
Convert texts to sequences of integers
padded_sequences = pad_sequences(sequences, maxlen=200) # Ensure all
sequences have the same length

# Prepare labels and convert 'Label' to binary format (0 for
Non-Spam, 1 for Spam)
train_data['Label'] = train_data['Label'].map({'Non-Spam': 0,
'Spam': 1})
test_data['Label'] = test_data['Label'].map({'Non-Spam': 0, 'Spam':
1})

# Split preprocessed data back into train and test sets
X_train, X_test = padded_sequences[:len(train_data)],
padded_sequences[len(train_data):]
y_train = train_data['Label'].values
y_test = test_data['Label'].values

# Adjust the initialization of the Sequential model to explicitly
define the input shape
model = Sequential()
model.add(Input(shape=(200,))) # Explicitly define the input shape
model.add(Embedding(input_dim=5000, output_dim=64)) # Now Embedding
layer without specifying input_length here
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=64,
validation_split=0.1)

```

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy:.2f}')
```

---

## *1. Preparación del Entorno y Carga de Datos*

### **A. Importación de Librerías**

```
# Import required libraries
import pandas as pd

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.layers import Input
import nltk

from nltk.corpus import stopwords
nltk.download('stopwords') # Download stopwords from NLTK
```

El proceso comienza con la importación de todas las librerías necesarias para la manipulación de datos, el procesamiento de texto y la construcción del modelo neuronal.

Se importa **Pandas** (**pd**) para manipular los datos; **NLTK** para el procesamiento del lenguaje natural; y de **TensorFlow/Keras**, se importan utilidades de preprocesamiento como **Tokenizer** (para convertir palabras a números) y **pad\_sequences** (para normalizar la longitud de las secuencias), además de las clases para la construcción de la red: **Sequential**, **Embedding**, **LSTM**, **Dense**, e **Input**. Adicionalmente, se requiere descargar las **stop words** de NLTK para limpiar el texto, lo cual se logra con un comando interno, por ejemplo, ejecutando **python -m nltk.downloader stopwords** en la terminal.

### **B. Carga del Dataset**

```
# Load datasets with the correct encoding
train_data = pd.read_csv('SMS_train.csv', encoding='latin1') # Load
training dataset
test_data = pd.read_csv('SMS_test.csv', encoding='latin1') # Load
testing dataset
```

A continuación, se cargan los conjuntos de datos de entrenamiento y de prueba en DataFrames de Pandas, prestando especial atención al formato.

Se cargan el fichero de entrenamiento (`SMS_train.csv`) y el de prueba (`SMS_test.csv`). En este caso, es crucial especificar el *encoding* como **latin1** al cargar los datos (`pd.read_csv`), ya que el *encoding* predeterminado (*UTF-8*) a menudo falla con estos tipos de *datasets*. Es importante tener claro el formato de los datos de origen para evitar problemas en el *parsing*. Posteriormente, se combinan ambos DataFrames en una única variable llamada **data** utilizando la función `pd.concat`, preparando un *dataset* unificado para el preprocesamiento de texto.

## C - Consolidación del Dataset:

Una vez cargados los *datasets* por separado, se utiliza la función `pd.concat` de Pandas para combinarlos en un único DataFrame llamado **data**:

```
# Combine train and test datasets for text preprocessing
data = pd.concat([train_data, test_data], ignore_index=True)
```

Propósito: Se concatenan (`pd.concat`) los DataFrames de prueba y entrenamiento. El argumento **ignore\_index=True** garantiza que el índice de las filas sea restablecido y consecutivo en el nuevo DataFrame consolidado. Este paso es esencial para aplicar las transformaciones de limpieza y tokenización al conjunto completo del texto de una sola vez.

## 2. *Preprocesamiento de Texto y Tokenización*

La fase de preprocesamiento es crítica para limpiar el texto y transformarlo en un formato numérico que la red neuronal pueda entender.

### A. Función de Preprocesamiento

```
# Text preprocessing function
def preprocess(text):
    # Convert to lowercase and tokenize
    tokens = nltk.word_tokenize(text.lower())
    # Remove stopwords
    stopwords = [word for word in tokens if word not in
stopwords.words('english')]
    return " ".join(stopwords)
```

Se define una función llamada **preprocess(text)** que realiza una serie de tareas de limpieza utilizando NLTK: primero, convierte el texto a **minúsculas** y luego lo **tokeniza** (divide en palabras) utilizando **nltk.word\_tokenize**. Seguidamente, **elimina las palabras vacías** (*stopwords*), que son palabras comunes (como "the", "a", "is") que no añaden valor semántico a la clasificación, utilizando el corpus de *stopwords* en inglés. Finalmente, la función une las palabras procesadas, devolviendo un texto limpio.

### B. Aplicación del Preprocesamiento

```
# Apply preprocessing to SMS texts
```



```
data['processed_text'] = data['Message_body'].apply(preprocess)
```

La función **preprocess** se aplica a la columna de texto original, llamada **Message\_body**, del DataFrame **data** utilizando el método **.apply(preprocess)**. El resultado de este preprocesamiento se asigna a una nueva columna del DataFrame llamada **processed\_text**. Así, cada fila de esta nueva columna contiene el texto limpio y listo para la tokenización.

### C. Tokenización y Relleno (*Padding*)

```
# Tokenization and Padding
tokenizer = Tokenizer(num_words=5000) # Initialize tokenizer with a
max number of words
tokenizer.fit_on_texts(data['processed_text']) # Fit tokenizer on
texts
sequences = tokenizer.texts_to_sequences(data['processed_text']) #
Convert texts to sequences of integers
padded_sequences = pad_sequences(sequences, maxlen=200) # Ensure all
sequences have the same length
```

Una vez que el texto está limpio, debe convertirse a secuencias de números (tokens) y normalizarse.

1. **Inicialización del Tokenizador:** Se inicializa un objeto **Tokenizer** de Keras, especificando **num\_words=5000**. Esto indica que el tokenizador solo tendrá en cuenta las **5.000 palabras más frecuentes** del *dataset* para la conversión.
2. **Ajuste del Tokenizador:** Se ajusta el tokenizador a los textos procesados (**tokenizer.fit\_on\_texts(data['processed\_text'])**). Esto

construye un **índice de palabras** basado en la frecuencia de las palabras en el *dataset*.

3. **Conversión a Secuencias:** La función `tokenizer.texts_to_sequences` convierte los textos procesados en secuencias de **números enteros** (*tokens*), donde cada palabra se reemplaza por su índice correspondiente en el vocabulario construido.
4. **Relleno de Secuencias (*Padding*):** Se utiliza la función `pad_sequences` para garantizar que todas las secuencias tengan la misma longitud. Con el parámetro `maxlen=200`, todas las secuencias se rellenarán con ceros o se truncarán para tener una longitud máxima de 200 *tokens*.

### 3. Definición, Entrenamiento y Evaluación del Modelo LSTM

#### A. Preparación de Etiquetas y División de Datos

```
# Prepare labels and convert 'Label' to binary format (0 for
Non-Spam, 1 for Spam)
train_data['Label'] = train_data['Label'].map({'Non-Spam': 0,
'Spam': 1})
test_data['Label'] = test_data['Label'].map({'Non-Spam': 0, 'Spam':
1})
```

```
# Split preprocessed data back into train and test sets
X_train, X_test = padded_sequences[:len(train_data)],
padded_sequences[len(train_data):]
y_train = train_data['Label'].values
```

```
y_test = test_data['Label'].values
```

En este punto, es necesario preparar las etiquetas y dividir el *dataset* preprocesado para las fases de entrenamiento y prueba.

Se preparan las etiquetas de los DataFrames originales (`train_data` y `test_data`) convirtiéndolas a un **formato binario**, donde **Non-Spam** se mapea a **0** y **Spam** se mapea a **1** mediante el método `.map()`. Finalmente, el *dataset* preprocesado se divide de nuevo en conjuntos de entrenamiento (`X_train`, `y_train`) y de prueba (`X_test`, `y_test`), utilizando la longitud de los DataFrames originales.

## B. Definición de la Arquitectura LSTM

```
# Adjust the initialization of the Sequential model to explicitly
define the input shape
model = Sequential()
model.add(Input(shape=(200,))) # Explicitly define the input shape
model.add(Embedding(input_dim=5000, output_dim=64)) # Now Embedding
layer without specifying input_length here
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))
```

El núcleo del proyecto es la definición de la arquitectura de la red neuronal **secuencial** basada en LSTM.

1. **Inicialización:** Se inicializa el modelo con `model = Sequential()`.
2. **Capa de Entrada:** Se añade una capa explícita de entrada (`model.add(Input(shape=(200,)))`). El parámetro `shape=(200,)` es crucial, ya que espera datos de entrada de longitud 200, coincidiendo con la longitud máxima definida en el *padding*.

3. **Capa de Incrustación (*Embedding*):** Se añade la capa **Embedding**. Esta capa transforma los *tokens* enteros de alta dimensionalidad en vectores densos de menor dimensionalidad. El **input\_dim** se establece en **5000** (el tamaño del vocabulario) y el **output\_dim** se establece en **64** (la dimensionalidad de los vectores de incrustación).
4. **Capa LSTM:** Se añade la capa **LSTM(64)** al modelo, que contiene 64 unidades. Esta capa es la encargada de capturar las dependencias secuenciales y a largo plazo en el texto.
5. **Capa Densa de Salida:** Se aplica una capa densa final (**Dense(1, activation='sigmoid')**) con una sola neurona. Se utiliza la función de activación **sigmoid** porque el problema es de **clasificación binaria**, generando una probabilidad entre 0 y 1 que indica si el mensaje es *spam* o no *spam*.

## C. Compilación, Entrenamiento y Evaluación

Se completa el ciclo de ML compilando, entrenando y evaluando el rendimiento del modelo.

### Compilación:

```
model.compile(optimizer='adam', loss='binary_crossentropy',  
metrics=['accuracy'])
```

- Se utiliza el optimizador **adam** (uno de los más eficientes). La función de pérdida **binary\_crossentropy** es la adecuada para tareas de clasificación binaria. Se monitoriza la métrica de **accuracy** (precisión) durante el proceso.

### Entrenamiento:

```
model.fit(X_train, y_train, epochs=10, batch_size=64,  
validation_split=0.1)
```

- El modelo se entrena durante **10 épocas** (ciclos completos de los datos). El **batch\_size** de **64** define el número de muestras que se propagan a través de la red antes de actualizar los parámetros. Se utiliza un **validation\_split** del 10% para monitorizar el rendimiento durante el entrenamiento.

### Evaluación:

```
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print(f'Test Accuracy: {accuracy:.2f}')
```

- El modelo se evalúa con los datos de prueba no vistos (X\_test y y\_test). El resultado de la terminal, por ejemplo, una precisión del **93%** (0.93...), indica un **alto rendimiento**. Este nivel de precisión sugiere un buen ajuste, sin sobreajuste ni subajuste, lo que hace el modelo apto para aplicaciones prácticas de filtrado de *spam*.

## 4. Testeo del Modelo con Texto Individual

Para validar el modelo en un escenario de uso real, se automatiza la limpieza y la predicción para un texto de entrada individual.

### A. Funciones de Predicción Automatizada

Se definen dos funciones esenciales para automatizar el proceso:

```
# Define function to preprocess text
def preprocess_input_text(text):
    tokens = nltk.word_tokenize(text.lower()) # Tokenize and convert
    to lowercase
    tokens = [word for word in tokens if word not in
stopwords.words('english')] # Remove stopwords
    return ' '.join(tokens) # Join tokens into a string
```

- **preprocess\_input\_text(text)**: Esta función automatiza el preprocesamiento del texto de entrada (tokenización, minúsculas, eliminación de *stopwords*), utilizando las funciones de NLTK.

```
# Define function to predict spam or not
def predict_spam_or_not(text):
    processed_text = preprocess_input_text(text) # Preprocess text
    sequence = tokenizer.texts_to_sequences([processed_text]) #
Convert text to sequence
    padded_sequence = pad_sequences(sequence, maxlen=200) # Apply
padding
    prediction = model.predict(padded_sequence) # Predict
    return "Spam" if prediction[0][0] > 0.5 else "No Spam" #
Interpret prediction
```

- **predict\_spam\_or\_not(text)**: Esta función gestiona el flujo completo de predicción. Llama a la función de preprocesamiento, convierte el texto a la secuencia numérica utilizando el **tokenizador original**, aplica el **padding** de longitud 200, y finalmente usa **model.predict(padded\_sequence)**. Interpreta la salida (probabilidad entre 0 y 1) como "Spam" si la predicción es mayor a 0.5 y como "No Spam" si es menor.

## B. Ejemplos de Clasificación

```
# Single text example
text_example = "Free entry in 2 a wkly comp to win FA Cup final tkts
21st May 2005. Text FA to 87121 to receive entry question(std txt
rate)T&C's apply 08452810075over18's"

# Predict if the text is spam or not spam
```

```
prediction_result = predict_spam_or_not(text_example)
print(f'The message: "{text_example}" is classified as:
{prediction_result}')
```

- **Ejemplo 1 (Spam Explícito):** Un mensaje que contiene términos típicos de sorteos y premios. El modelo, tras el análisis, clasifica correctamente el mensaje como **Spam**.

```
# Nuevo mensaje de ejemplo para clasificar
text_example = "Meet me at the mall tomorrow. I found something
you'll love!"
```

```
# Predecir si el nuevo texto es spam o no
prediction_result = predict_spam_or_not(text_example)
print(f'The message: "{text_example}" is classified as:
{prediction_result}')
```

- **Ejemplo 2 (No Spam Genuino):** Un mensaje personal y coherente (ej., "Meet me at the mall tomorrow..."). El modelo clasifica correctamente el mensaje como **No Spam**, demostrando su capacidad para diferenciar el lenguaje conversacional.

```
# Nuevo mensaje de ejemplo que probablemente sea spam
text_example = "Congratulations! You've been selected for an
exclusive offer. Claim your $1000 gift card now by visiting our
site."
```

```
# Predecir si el nuevo texto es spam o no
prediction_result = predict_spam_or_not(text_example)
print(f'The message: "{text_example}" is classified as:
{prediction_result}')
```

- **Ejemplo 3 (Spam Obvio):** Un mensaje con felicitaciones y una oferta exclusiva. El modelo clasifica correctamente el mensaje como **Spam**.

El modelo, aunque simple, demuestra un **buen gestor de posible spam** o correo malicioso. Esta plantilla de código puede ser adaptada fácilmente a cualquier arquitectura Python para una gestión preliminar de amenazas.

## 5. Conclusiones del Desarrollo

Este ejercicio se centra en el desarrollo de un sistema avanzado de detección de amenazas por correo electrónico utilizando redes neuronales recurrentes, específicamente la arquitectura **LSTM** (*Long Short-Term Memory*). El objetivo ha sido clasificar los *emails* en *spam* o no *spam* basándonos en su contenido textual a través de la importación de librerías clave como **Pandas** y **TensorFlow**.

El ejercicio ha demostrado que la tarea de **preprocesamiento de texto**, que incluye normalizar, eliminar *stopwords*, **tokenizar** y **rellenar** las secuencias, es esencial para preparar los datos. La estructura de la LSTM se ajusta específicamente para capturar las **dependencias a largo plazo y las secuencias** en datos de texto.

Al compilar el modelo con el optimizador **Adam** y la función de pérdida **Binary CrossEntropy**, se obtiene la configuración adecuada para tareas de clasificación binaria. La aplicación de este enfoque ofrece un método robusto y sofisticado para filtrar *emails*, protegiendo a los usuarios de posibles fraudes, *phishing* y *malware*. El uso de una LSTM permite al modelo aprender de secuencias de palabras y contextos complejos, lo que **supera a los métodos tradicionales de detección de spam** que se basan únicamente en la coincidencia de palabras clave o reglas fijas.

## Asegurando la Autenticación de Usuarios con Inteligencia Artificial



En un mundo donde la seguridad digital es más crítica que nunca, la **Inteligencia Artificial (IA)** ofrece soluciones avanzadas que van más allá de las contraseñas y los métodos de autenticación habituales. La IA no solo mejora la seguridad, sino que también proporciona una experiencia de usuario más fluida y adaptativa.

La autenticación de usuarios es el primer paso para proteger los sistemas y la información sensible de accesos no autorizados. Si bien tradicionalmente se ha confiado en métodos basados en algo que el usuario "sabe" (como una contraseña), la IA nos permite evolucionar hacia métodos basados en algo que el usuario "**es**" (mediante la biometría) y algo que el usuario "**hace**" (a través de la biometría del comportamiento). Este enfoque dual no solo aumenta la seguridad, sino que también mejora la experiencia del usuario al minimizar las fricciones durante el proceso.

---

## *1. Evolución de la Autenticación: Biometría con IA*

La **autenticación biométrica** utiliza características físicas únicas para verificar la identidad de un usuario, incluyendo huellas dactilares, el iris, la voz o el **reconocimiento facial**. La Inteligencia Artificial mejora significativamente estos sistemas.

- **Reconocimiento Facial Avanzado:** Los algoritmos de **aprendizaje profundo** (*Deep Learning*) permiten que los sistemas de reconocimiento facial analicen **miles de puntos de referencia** en una cara. Esto hace que la tecnología sea extremadamente precisa y difícil de engañar con métodos de suplantación como fotos o máscaras.
- **Aprendizaje Continuo y Adaptación:** La IA permite que estos sistemas se **adapten a cambios en la apariencia del usuario** (como el envejecimiento, el uso de gafas o el cambio de peinado). Un sistema de seguridad bancario, por ejemplo, puede utilizar reconocimiento facial con IA para permitir el acceso a cajeros automáticos. Gracias al **aprendizaje continuo**, la IA se ajusta y mejora con cada uso, manteniendo la fiabilidad a largo plazo.

## 2. *Biometría del Comportamiento (Behavioral Biometrics)*

La **biometría del comportamiento** representa un avance revolucionario en la autenticación, ya que se centra en cómo interactúa el usuario con un dispositivo o aplicación.

- **Análisis de Patrones Únicos:** Esta tecnología utiliza algoritmos de IA para aprender **patrones únicos de comportamiento** del usuario, tales como la forma en que está **tecleando** (ritmo, velocidad y presión), cómo mueve el **ratón**, o la manera en que **interactúa con la interfaz** de cualquier aplicación.
- **Seguridad Personalizada:** Estos patrones son extremadamente difíciles de imitar, lo que ofrece un nivel de seguridad muy **personalizado y eficiente**.
- **Ejemplo Práctico en Banca:** Siguiendo el ejemplo de una aplicación bancaria en línea, si el sistema detecta una **anomalía** en el comportamiento habitual del teclear o en la forma de navegar del usuario, podría solicitar automáticamente una **autenticación adicional**. Esta autenticación podría ser un reconocimiento facial o un *token* de seguridad enviado al teléfono móvil del usuario para confirmar su identidad antes de permitir la transacción.

## 3. *Cifrado, Desafíos y Enfoque Multifactor*

A pesar de las grandes ventajas de la biometría y la IA, su implementación segura presenta desafíos que requieren medidas adicionales de protección.

### A. **Desafíos de Implementación y Privacidad**

La implementación de la autenticación basada en IA enfrenta desafíos cruciales, incluyendo:

- **Privacidad de Datos Biométricos:** Existen preocupaciones legítimas sobre la privacidad y el uso de datos biométricos sensibles.
- **Sistemas de Respaldo (*Failover*):** Se requiere la necesidad de sistemas de respaldo robustos en caso de fallos del sensor biométrico o errores del algoritmo.

Para afrontar estas cuestiones, es esencial adoptar un **enfoque de privacidad por diseño** (*Privacy by Design*), asegurando que los datos biométricos estén protegidos con las mejores prácticas de **cifrado y almacenamiento seguro**.

## **B. Autenticación Multifactor y Redundancia**

La **autenticación multifactor** ofrece una capa adicional de seguridad y redundancia al combinar varios métodos de verificación. Puede implicar la **combinación de biometría y tokens** (algo que el usuario "es" y algo que el usuario "tiene").

La **ciberseguridad bancaria** ilustra perfectamente la convergencia de estas tecnologías para proteger el acceso de los usuarios:

<IMAGEN del grafico que se describe a continuación>

El diagrama de flujo muestra cómo un **Sistema de Seguridad Bancario** se ramifica en tres pilares: **Autenticación Biométrica con IA** (ej. Reconocimiento Facial), **Biometría de Comportamiento** (ej. Análisis de Patrones de Tecleo), y **Autenticación Multifactor** (ej. Combinación de Biometría y *Tokens*). Todos estos flujos convergen en el **Cifrado y Seguridad de Datos**, que a su vez dirige la **Implementación y Desafíos** para llegar a la **Conclusión de Seguridad Mejorada**. Se destaca la importancia de cifrar los datos en la base del sistema.

## *Conclusiones*

La implementación de tecnologías de autenticación basadas en IA, especialmente la **autenticación biométrica** y la **biometría del comportamiento**, representa un avance significativo en la protección de usuarios en cualquier tipo de servicio, no solo el bancario.

Estas tecnologías no solo ofrecen un **nivel de seguridad superior** a los métodos tradicionales al utilizar características únicas del usuario, sino que también **mejoran la experiencia al acceder a los servicios** al minimizar la fricción. Al combinar estos métodos avanzados de autenticación con prácticas robustas de **cifrado y técnicas clásicas de seguridad de los datos**, las organizaciones pueden enfrentarse de

forma más efectiva a las amenazas digitales emergentes, garantizando al mismo tiempo la privacidad y la confianza de los clientes.